Day 1: Docker Fundamentals – Containers, Images, and Dockerfiles

Welcome to Day 1 of the Containers & Kubernetes workshop! Today, we will explore **Docker**, the industry-standard container platform. By the end of the day, you will have a deep understanding of container concepts and practical skills to:

- Differentiate containers from virtual machines.
- Understand Docker's architecture (client, daemon, registries).
- Manage Docker images and containers.
- Write custom `Dockerfile`s and build efficient images.
- Use volumes for persistent data.
- Connect containers via custom networks.
- Run multi-container applications with Docker Compose.

This day you'll learn by doing. Execute every command, observe the outputs, and you'll master Docker by evening.

Prerequisites

Ensure Docker is installed and running on your machine. Verify with:

```
docker --version
docker info
```

If Docker is not installed, refer to the pre-requisites setup guide provided before the workshop.

09:00 – 10:30 – Conceptual Session: Container Basics

1.1 What are Containers?

Analogy

Shipping containers revolutionised global trade by standardising cargo – any container fits on any ship, train, or truck. Similarly, **software containers** standardise application deployment: they bundle an application with its entire runtime environment (code, libraries, dependencies) into a single package that runs consistently anywhere.

Containers vs Virtual Machines

Virtual machines (VMs) virtualise hardware and run a full guest operating system. Containers virtualise the operating system and share the host's kernel. This makes containers far more lightweight.

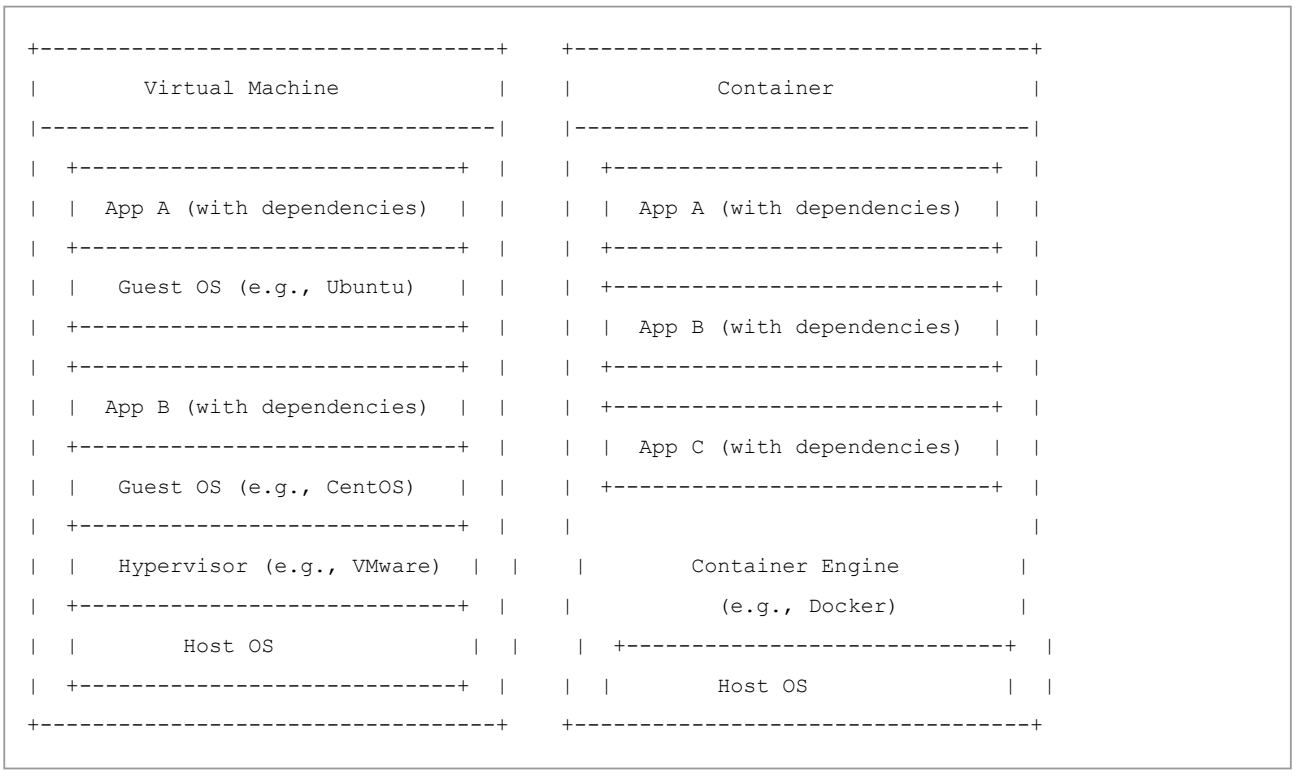
Feature	Virtual Machine	Container
OS	Each VM includes a full guest OS	Shares the host OS kernel
Size	Gigabytes (GB)	Megabytes (MB)
Startup time	Minutes	Seconds
Isolation	Strong (hardware-level)	Process-level (namespaces & cgroups)
Resource overhead	High (hypervisor, multiple OS copies)	Low (only the app and its dependencies)

Why Containers?

- **Portability**: Run the same container on a developer's laptop, a test server, or in the cloud.
- **Efficiency**: Higher density – more applications on the same hardware.
- **Speed**: Rapid start/stop accelerates development and scaling.
- **Consistency**: "It works on my machine" becomes a thing of the past.

Visual Comparison

Below is a simplified diagram of VMs versus Containers:



Demo – Your First Container

Run the famous "hello-world" container to confirm everything works:

```
docker run hello-world
```

You'll see a welcome message explaining that Docker works correctly. Behind the scenes, Docker:

1. Checked locally for the `hello-world` image (not found).
2. Pulled it from Docker Hub (a public registry).
3. Created a container from that image.
4. Ran the container, which printed the message and exited.

1.2 Docker Architecture

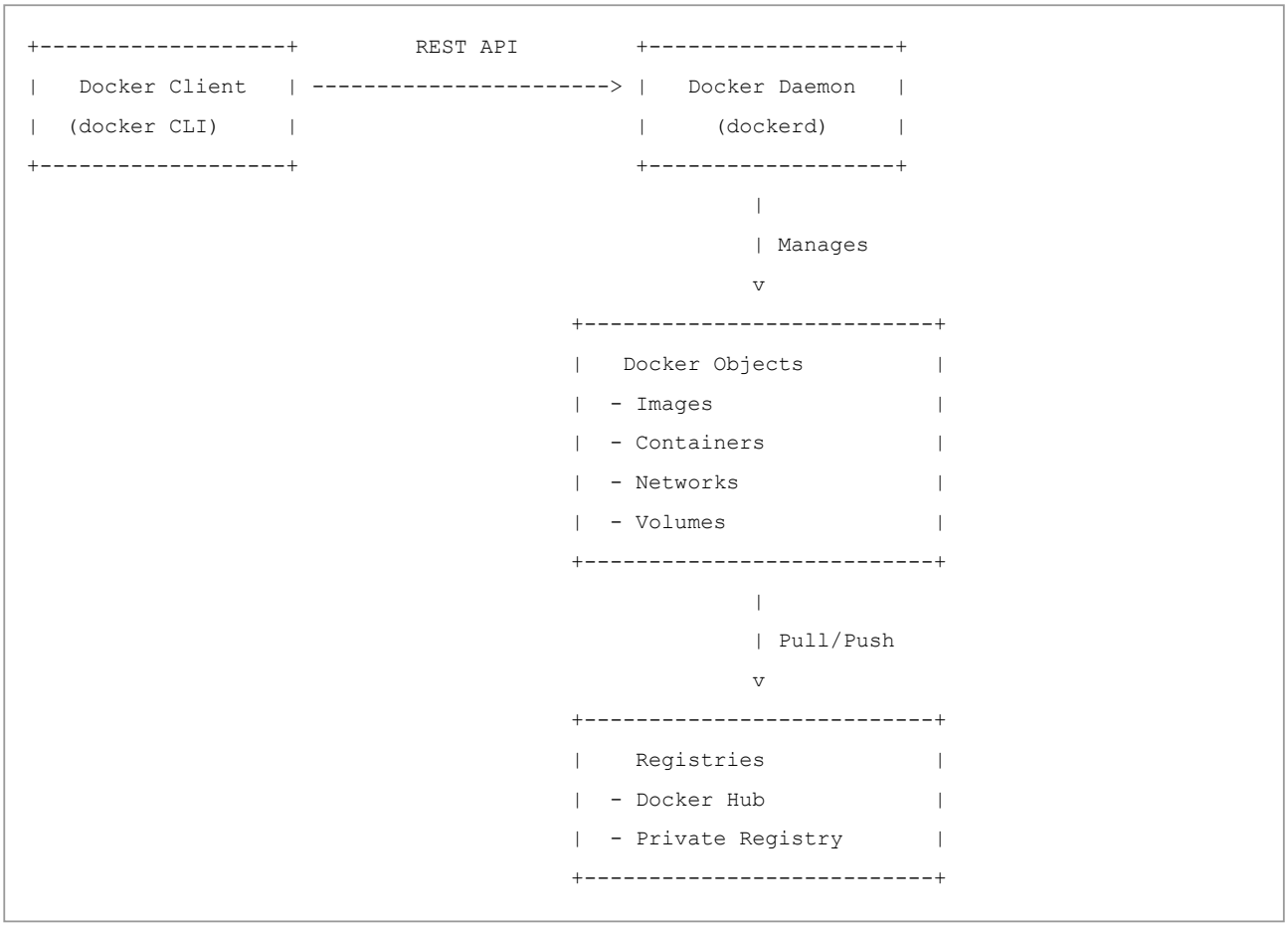
Docker uses a client-server architecture:

- **Docker client** (`docker` command) – the primary way users interact with Docker. It sends commands to the daemon.
- **Docker daemon** (`dockerd`) – a persistent background process that manages Docker objects: images, containers, networks, volumes.
- **Docker registries** – repositories for Docker images. Docker Hub is the default public registry. You can also run private registries.

Communication

The client and daemon can run on the same machine or communicate remotely via REST API, UNIX sockets, or a network interface.

Diagram: Docker Architecture



Key Objects

- **Image** – a read-only template with instructions for creating a container. You can build your own or use images created by others.
- **Container** – a runnable instance of an image. You can create, start, stop, move, or delete a container using the Docker API or CLI.
- **Registry** – stores images. Docker Hub is the default.

Demo

```
docker images          # list downloaded images
docker pull nginx      # download the nginx image from Docker Hub
docker run nginx       # run a container from the nginx image (foreground)
docker ps              # list running containers
docker ps -a           # list all containers (including stopped)
```

1.3 Dockerfile Basics

A `Dockerfile` is a text file containing a series of instructions to assemble an image. Each instruction creates a **layer** in the image. Layers are cached and reused, making builds faster.

Common Dockerfile Instructions

Instruction	Description	Example
`FROM`	Sets the base image (must be the first instruction).	`FROM ubuntu:22.04`
`RUN`	Executes a command during build (e.g., install packages).	`RUN apt update && apt install -y curl`
`COPY`	Copies files or directories from the host into the image.	`COPY app.py /app/`
`ADD`	Similar to `COPY` but can also handle remote URLs and tar extraction.	`ADD https://example.com/file.tar.gz /tmp/`
`WORKDIR`	Sets the working directory for subsequent `RUN`, `CMD`, `ENTRYPOINT`.	`WORKDIR /app`
`CMD`	Provides defaults for an executing container (can be overridden).	`CMD ["python", "app.py"]`
`ENTRYPOINT`	Configures a container to run as an executable (harder to override).	`ENTRYPOINT ["top", "-b"]`
`EXPOSE`	Informs Docker that the container listens on the specified port (documentation only).	`EXPOSE 80`
`ENV`	Sets environment variables.	`ENV NODE_ENV=production`

Layer Caching

Docker builds images layer by layer. If a layer hasn't changed, Docker reuses the cached version. To optimise caching, order instructions from least frequently changed to most frequently changed (e.g., copy `requirements.txt` before copying the entire application).

Diagram: Image Layers

```
+---------------------+
|   Container Layer   |  (writable)
+---------------------+
|   Image Layer (4)   |  (read-only)  ← e.g., CMD
+---------------------+
|   Image Layer (3)   |  (read-only)  ← e.g., COPY app.py
+---------------------+
|   Image Layer (2)   |  (read-only)  ← e.g., RUN pip install
+---------------------+
|   Image Layer (1)   |  (read-only)  ← e.g., FROM python
+---------------------+
|   Base Layer        |  (from the base image)
+---------------------+
```

Each layer is identified by a hash. When you change a file, only the layers above it need to be rebuilt.

Demo – Simple Python App Dockerfile

Create a file named `Dockerfile`:

```
FROM python:3.9-slim
WORKDIR /app
COPY app.py .
RUN pip install flask
CMD ["python", "app.py"]
```

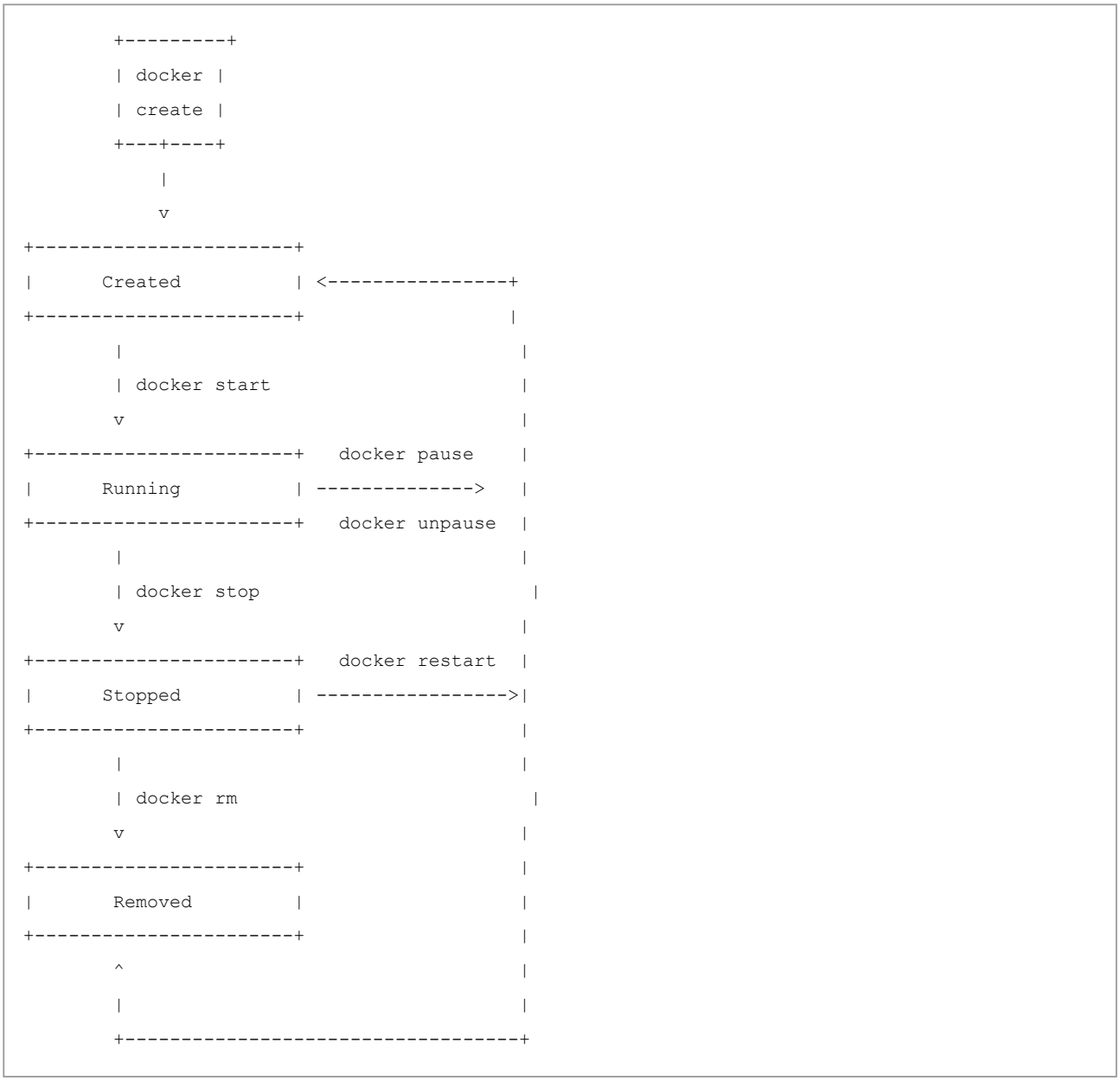
Then build and run:

```
docker build -t my-python-app .
docker run -p 5000:5000 my-python-app
```

1.4 Container Lifecycle

A container can be in one of several states: **created**, **running**, **paused**, **stopped**, **deleted**.

State Transition Diagram



Essential Commands

Command	Description
`docker run <image>`	Create and start a container from an image.
`docker start <container>`	Start a stopped container.
`docker stop <container>`	Gracefully stop a running container (SIGTERM).
`docker kill <container>`	Force stop a container (SIGKILL).
`docker pause <container>`	Suspend all processes in a container.
`docker unpause <container>`	Resume a paused container.
`docker rm <container>`	Remove a stopped container.
`docker logs <container>`	Fetch the logs of a container.
`docker exec <container> <cmd>`	Run a command inside a running container.

Port Publishing

By default, containers are isolated from the host network. To expose a container's port, use `-p` (or `--publish`):

```
docker run -d -p 8080:80 --name my-nginx nginx
```

- `-d`: detached mode (background).
- `-p 8080:80`: map host port 8080 to container port 80.
- `--name`: assign a name to the container.

Now visit `http://localhost:8080` – you'll see the Nginx welcome page.

Environment Variables

Pass configuration via `-e`:

```
docker run -e MY_VAR=hello alpine env
```

10:45 – 12:30 – Hands-on Lab 1: Docker Basics

Perform each exercise step by step. Keep your terminal open.

Exercise 1.1 – Run Your First Container

```
docker run hello-world
```

Expected output: A welcome message explaining that your installation appears to be working correctly.

```
docker run -d -p 8080:80 --name my-nginx nginx
```

- `-d`: runs in background.
- Check with `docker ps` to see it running.

```
curl http://localhost:8080
```

You should see the Nginx default HTML. Alternatively, open a browser.

```
docker logs my-nginx
```

Shows access logs.

```
docker stop my-nginx
docker rm my-nginx
```

Exercise 1.2 – Work with Images

```
docker pull ubuntu:22.04
docker images
```

```
docker run -it ubuntu:22.04 bash
```

Inside the container:

```
apt update
apt install -y curl
curl --version
exit
```

```
docker ps -a
# Note the container ID of the exited ubuntu container
docker commit <container-id> my-ubuntu:with-curl
docker images | grep my-ubuntu
docker run -it my-ubuntu:with-curl curl --version
```

Exercise 1.3 – Write a Simple Dockerfile

```
mkdir docker-exercise3 && cd docker-exercise3
```

Create `Dockerfile`:

```
FROM alpine:latest
RUN apk add --no-cache curl
CMD ["sh", "-c", "echo 'Container ready!' && sleep infinity"]
```

Build and run:

```
docker build -t my-alpine .
docker run -d --name alpine-test my-alpine
docker exec alpine-test curl --version
```

Exercise 1.4 – Port Publishing and Environment Variables

```
mkdir html
echo "<h1>Hello from Docker</h1>" > html/index.html
docker run -d -p 8081:80 -v $(pwd)/html:/usr/share/nginx/html --name web nginx
curl http://localhost:8081
```

Environment variable example:

```
docker run --rm -e GREETING="Hello Workshop" alpine sh -c 'echo $GREETING'
```

Exercise 1.5 – Container Cleanup

```
docker stop $(docker ps -q)
docker rm $(docker ps -aq)
docker rmi my-alpine my-ubuntu:with-curl
docker system prune -f   # optional
```

13:30 – 15:00 – Conceptual Session: Advanced Docker

2.1 Multi-stage Builds

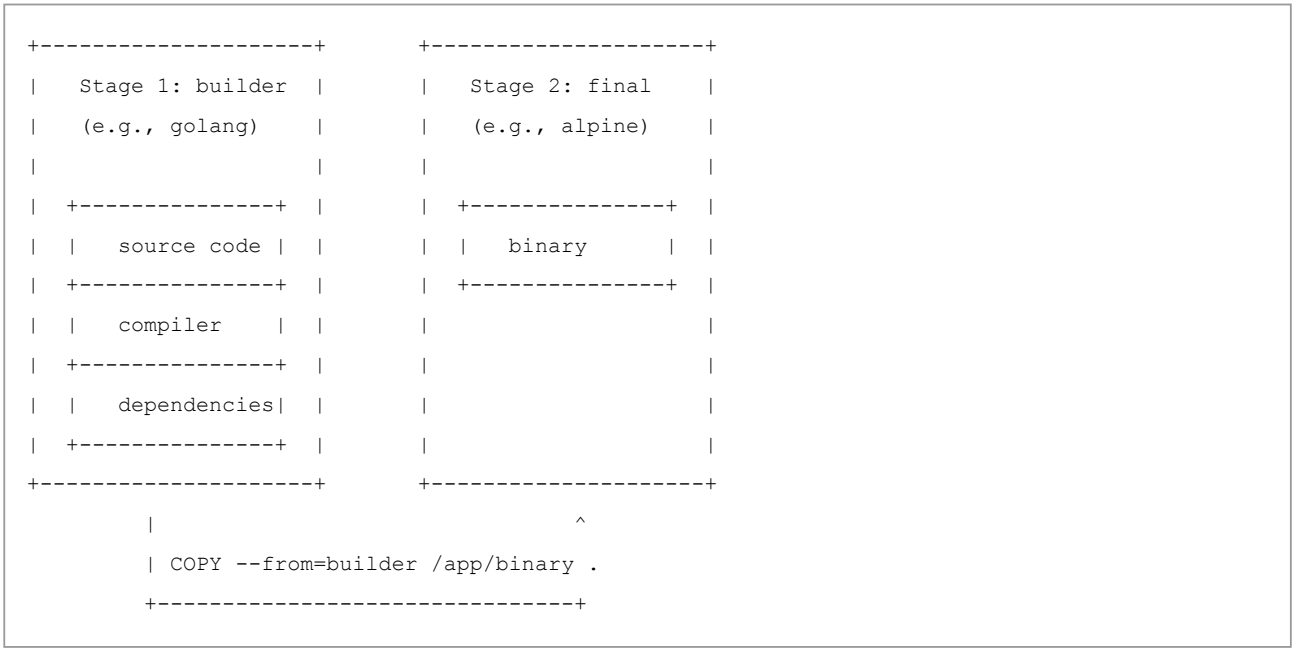
Why Multi-stage?

When building applications (especially compiled languages like Go, C, or Java), you need build tools and dependencies that are unnecessary at runtime. Multi-stage builds let you use one image for building and a second, minimal image for running – copying only the artifacts.

How It Works

You define multiple `FROM` statements. Each `FROM` begins a new stage. You can selectively copy files from previous stages using `COPY --from=<stage-name>`. Only the last stage's layers are kept in the final image.

Diagram: Multi-stage Build



Example – Go Application

```
# Build stage
FROM golang:1.20 AS builder
WORKDIR /app
COPY main.go .
RUN go build -o myapp main.go

# Final stage
FROM alpine:latest
WORKDIR /root/
COPY --from=builder /app/myapp .
CMD ["./myapp"]
```

The final image contains only the compiled binary, no Go compiler or build tools – significantly smaller.

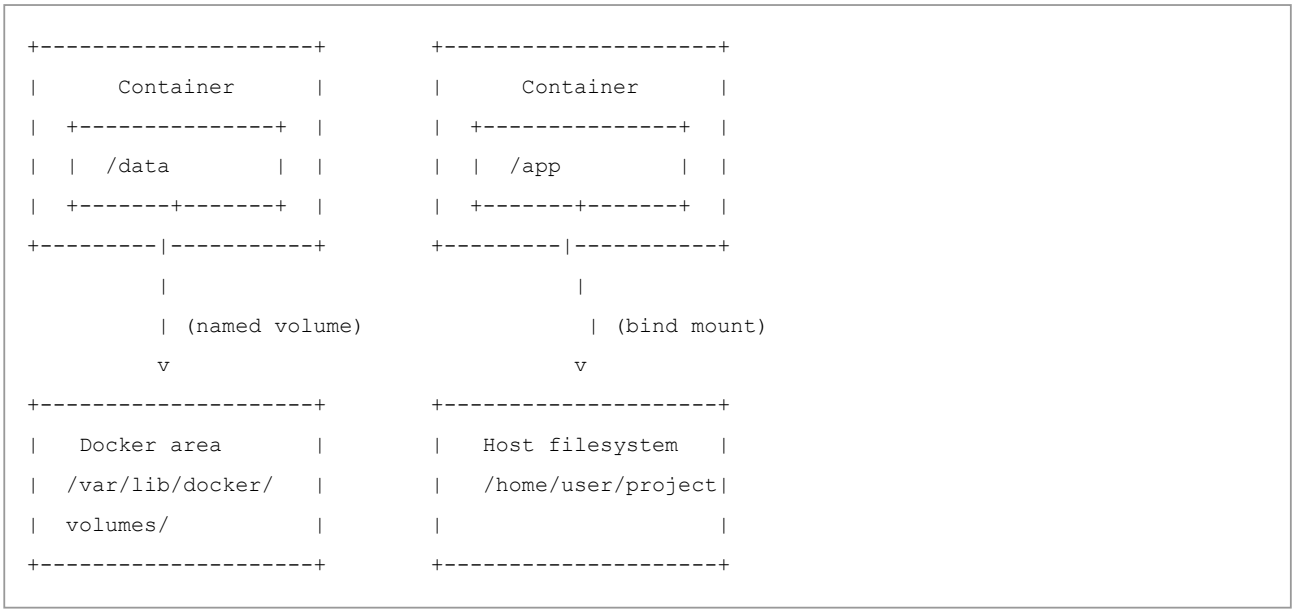
2.2 Docker Volumes

Containers are ephemeral by design – when you delete a container, its data disappears. **Volumes** provide persistent storage independent of container lifecycle.

Types of Mounts

Type	Description	Managed by	Use case
Named volume	Stored in Docker's area (`/var/lib/docker/volumes/`). Referenced by name.	Docker	Persistent data (databases, configs).
Bind mount	Maps any host directory into the container.	User	Sharing source code, configuration files.
tmpfs mount	Stored in memory only, never written to filesystem.	Docker	Sensitive temporary data.

Diagram: Volumes vs Bind Mounts



Demo – MySQL with Named Volume

```
docker volume create mysql-data
docker run -d --name mysql-db -v mysql-data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=secret mysql:8
```

Even after removing the container, the data persists in the volume.

2.3 Docker Networks

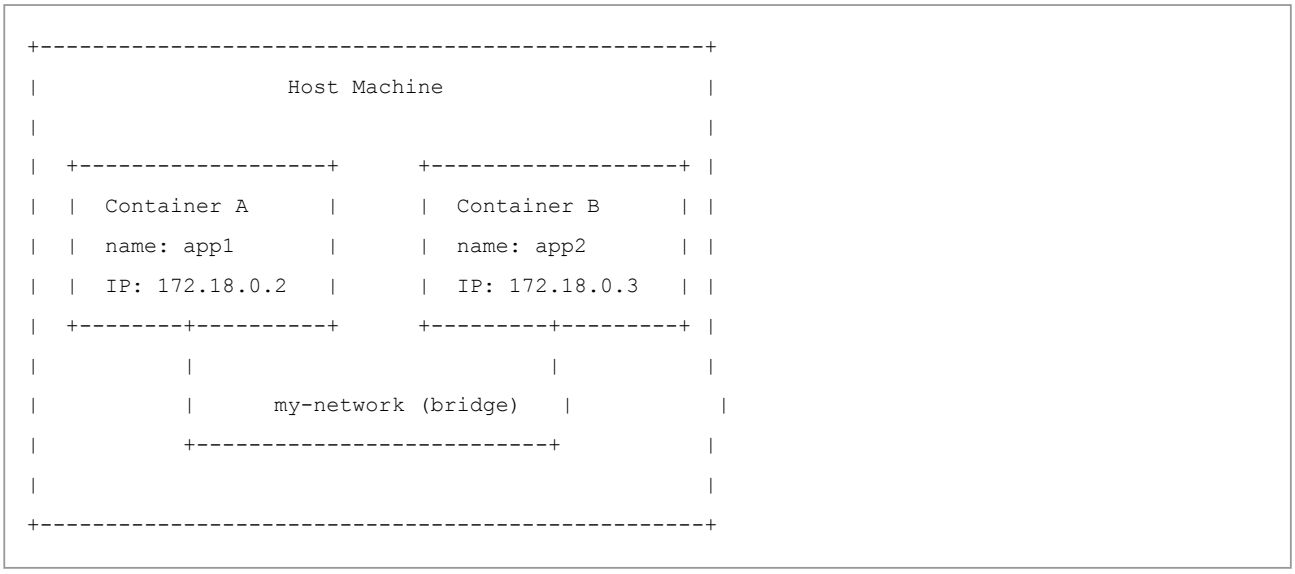
Docker provides networking so containers can communicate. The default network types are:

Network Driver	Description
bridge	Default network. Containers on the same bridge can communicate via IP.
host	Container shares the host's network stack (no isolation).
none	Container has no network interfaces.
overlay	For connecting containers across multiple Docker hosts (swarm mode).

Custom Networks

You can create your own bridge networks. Containers on the same user-defined bridge can resolve each other by container name (automatic DNS). This is not possible on the default bridge.

Diagram: Bridge Network with DNS



Containers can ping each other using names: `ping app2` from `app1`.

Demo

```
docker network create my-network
docker run -d --name app1 --network my-network alpine sleep infinity
docker run -d --name app2 --network my-network alpine sleep infinity
docker exec app1 ping app2   # succeeds
```

2.4 Docker Compose

Docker Compose is a tool for defining and running multi-container applications. You write a YAML file to configure your application's services, networks, and volumes. Then, with a single command, you create and start all services.

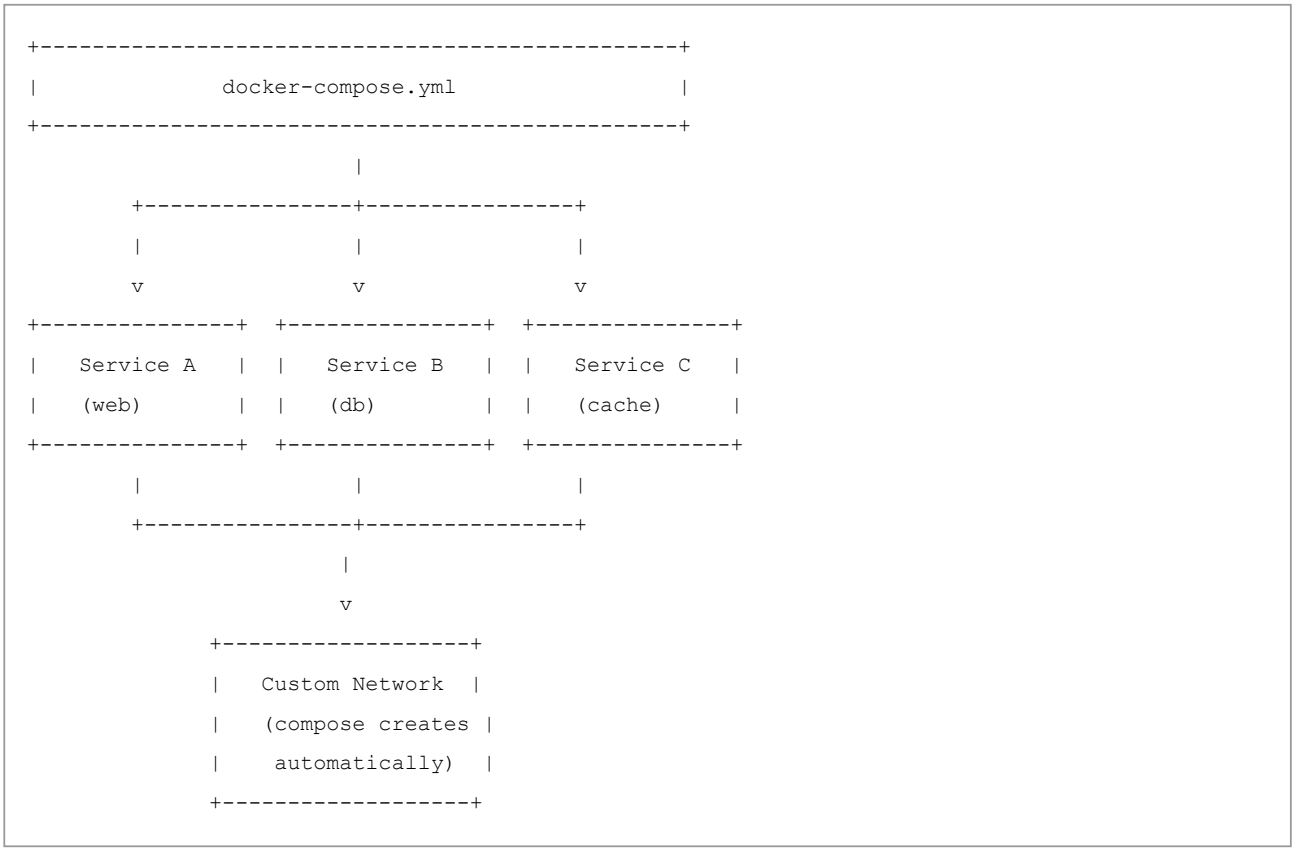
Compose File Structure

```
version: '3'          # Compose file version
services:             # Define each container
  web:
    image: nginx
    ports:
      - "8080:80"
  db:
    image: mysql:8
    volumes:
      - db_data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: secret
volumes:              # Named volumes
  db_data:
networks:             # Custom networks (optional)
  mynet:
```

Key Commands

- `docker-compose up -d` – create and start all services in background.
- `docker-compose down` – stop and remove containers, networks (by default, not volumes).
- `docker-compose logs` – view logs.
- `docker-compose ps` – list containers.

Diagram: Compose Multi-Service Application



Demo – WordPress + MySQL (as shown earlier)

15:15 – 16:45 – Hands-on Lab 2: Advanced Docker

Follow these exercises to reinforce the advanced concepts.

Exercise 2.1 – Multi-stage Dockerfile

Goal: Build a minimal Flask application using multi-stage builds.

```
mkdir flask-multistage && cd flask-multistage
```

Create `app.py`:

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Flask inside a multi-stage Docker container!"
```

Create `requirements.txt`:

```
Flask==2.3.3
```

Create `Dockerfile`:

```
# Builder stage
FROM python:3.9-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN python -m venv /venv && /venv/bin/pip install --no-cache-dir -r requirements.txt

# Final stage
FROM python:3.9-slim
WORKDIR /app
COPY --from=builder /venv /venv
COPY app.py .
ENV PATH="/venv/bin:$PATH"
EXPOSE 5000
CMD ["python", "app.py"]
```

Build and run:

```
docker build -t flask-multistage .
docker run -d -p 5000:5000 --name flask-app flask-multistage
curl http://localhost:5000
docker images | grep flask-multistage   # check small size
docker stop flask-app && docker rm flask-app
```

Exercise 2.2 – Persistent Volume with MySQL

```
docker volume create mysql-data
docker run -d --name mysql-db -v mysql-data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=secret mysql:8
docker exec -it mysql-db mysql -p
# (inside MySQL) create database testdb; show databases; exit
docker stop mysql-db && docker rm mysql-db
# Start a new container with same volume
docker run -d --name mysql-db2 -v mysql-data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=secret mysql:8
docker exec -it mysql-db2 mysql -p -e "SHOW DATABASES;"   # testdb should appear
docker stop mysql-db2 && docker rm mysql-db2
docker volume rm mysql-data
```

Exercise 2.3 – Custom Network

```
docker network create my-network
docker run -d --name app1 --network my-network alpine sleep infinity
docker run -d --name app2 --network my-network alpine sleep infinity
docker exec app1 ping app2
docker network inspect my-network
docker stop app1 app2 && docker rm app1 app2
docker network rm my-network
```

Exercise 2.4 – Docker Compose WordPress

```
mkdir wordpress-compose && cd wordpress-compose
```

Create `docker-compose.yml`:

```
version: '3'
services:
  db:
    image: mysql:8
    volumes:
      - db_data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: wordpress
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: wordpress
  wordpress:
    depends_on:
      - db
    image: wordpress:latest
    ports:
      - "8000:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: wordpress
volumes:
  db_data:
```

Run:

```
docker-compose up -d
docker-compose ps
# open browser at http://localhost:8000
docker-compose logs -f
# Ctrl+C to stop following logs
docker-compose down -v   # stops and removes volumes
```

Exercise 2.5 – Cleanup Everything

```
docker stop $(docker ps -q)
docker rm -f $(docker ps -aq)
docker rmi flask-multistage my-alpine my-ubuntu:with-curl   # if desired
docker system prune -a --volumes
```

Daily Recap & Troubleshooting

Key Takeaways

- Containers are lightweight, portable, and share the host kernel.
- Docker images are built in layers; caching speeds up builds.
- Multi-stage builds produce smaller production images.
- Volumes persist data beyond container lifecycles.
- Custom networks enable container-to-container communication by name.
- Docker Compose orchestrates multi-container applications from a single YAML file.

Common Issues & Solutions

Problem	Likely Cause	Solution
`docker: command not found`	Docker not installed or not in PATH.	Install Docker; add user to `docker` group and log out/in.
`permission denied` when connecting to daemon	User not in `docker` group.	`sudo usermod -aG docker $USER` then log out/in.
Port already in use	Another process is using that host port.	Change the host port (e.g., `8081` instead of `8080`).
Container exits immediately	The command finishes quickly (e.g., bash with no input).	Use `-it` for interactive or run a long-running process.
Volume not persisting	Using bind mount with incorrect path.	Use absolute paths or named volumes.
Compose services can't reach each other	Not using service names as hostnames.	In the same Compose file, use the service name (e.g., `db`) as hostname.
`docker build` fails with "no such file"	File not in build context.	Ensure files are in the same directory as the Dockerfile (or subdirectory).

Connection to Previous Modules

- **Scripting (Days 36–40)**: You used Bash commands throughout these exercises; writing Dockerfiles is similar to writing scripts.
- **Infrastructure (Days 45–47)**: Networking (IP, DNS) maps directly to Docker networks; storage concepts apply to volumes.
- **Security (Days 49–50)**: Principle of least privilege will be applied to containers later (running as non-root, using secrets).

Congratulations! You have completed Day 1 and now possess a strong Docker foundation. Tomorrow we will step into Kubernetes and learn how to orchestrate containers at scale. Keep your Docker environment ready for Day 2!